

RAG SYSTEM ENGINEERING KT GUIDE

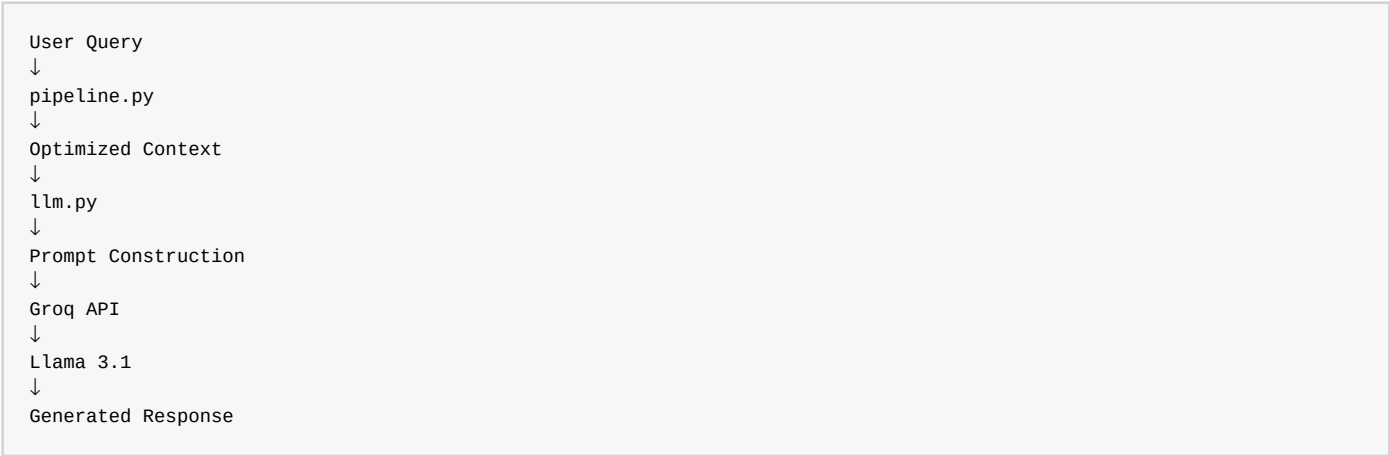
Module: llm.py (LLM Generation Layer)

Layer	Responsibility
Environment Layer	Loads secure API variables
Client Layer	Initializes Groq client
Prompt Engineering Layer	Builds grounded prompts
Streaming Layer	Real-time token generation
Generation Layer	Returns final LLM answer

0. Learner Workflow

Step	What Learner Should Do
1	Open llm.py
2	Understand environment loading
3	Learn secure API initialization
4	Trace prompt engineering flow
5	Understand streaming architecture
6	Learn generator-based streaming
7	Compare streaming vs standard generation
8	Trace final answer generation
9	Use FULL FILE REFERENCE for indentation clarity
10	Move to FastAPI app.py

1. Position in Overall Architecture



2. Why llam.py Exists

This module becomes the final AI generation layer.

It solves:

- prompt engineering
- grounded answer generation
- hallucination reduction
- streaming responses
- LLM API communication
- synchronous response generation

3. Exact Step-by-Step Code Walkthrough

STEP 1 — Import OS Module

```
import os
```

Used for secure environment variable access.

STEP 2 — Import dotenv

```
from dotenv import load_dotenv
```

Loads .env variables securely.

STEP 3 — Import Groq SDK

```
from groq import Groq
```

Provides Groq LLM API client.

STEP 4 — Load Environment Variables

```
load_dotenv()
```

Loads API secrets from .env into environment memory.

STEP 5 — Initialize Groq Client

```
client = Groq(  
    api_key=os.getenv("GROQ_API_KEY")  
)
```

Creates reusable singleton LLM client.

STEP 6 — Streaming Generation Function

```
def generate_stream_answer(  
    query: str,  
    context: str  
):
```

Enables real-time token streaming.

STEP 7 — Prompt Engineering

```
prompt = f"""  
You are a strict assistant.  
...  
"""
```

Grounded prompting reduces hallucination risk.

STEP 8 — Streaming Completion Call

```
stream = client.chat.completions.create(  

```

Starts real-time streaming session.

STEP 9 — Enable Streaming

```
stream=True
```

Tokens arrive incrementally instead of waiting for full completion.

STEP 10 — Iterate Over Token Chunks

```
for chunk in stream:
```

Processes streaming token chunks progressively.

STEP 11 — Yield Generated Tokens

```
yield chunk.choices[0].delta.content
```

Uses Python generators for memory-efficient streaming.

STEP 12 — Standard Generation Function

```
def generate_answer(  
    query: str,  
    context: str  
):
```

Creates full synchronous answer generation.

STEP 13 — Standard Completion Call

```
response = client.chat.completions.create(  

```

Executes full non-streaming generation request.

STEP 14 — Final Response Extraction

```
return response.choices[0].message.content
```

Returns final generated answer string.

4. Streaming vs Non-Streaming Architecture

Feature	Streaming	Non-Streaming
User Experience	Better	Normal
Latency Perception	Lower	Higher
Complexity	Higher	Lower
Chat Applications	Excellent	Good
API Usage	Optional	Common

5. Security Engineering Concepts

Concept	Why Important
.env Usage	Protects API secrets
os.getenv()	Secure environment access
No Hardcoded Keys	Prevents credential leakage
Grounded Prompting	Reduces hallucination risk

6. FULL llm.py FILE REFERENCE (Indentation Guide)

This section helps learners:

- understand execution hierarchy
- manually implement the file correctly
- avoid indentation mistakes
- understand streaming flow deeply

```
import os
from dotenv import load_dotenv
from groq import Groq
```

```
# Load environment variables
load_dotenv()

# Initialize Groq client
client = Groq(
    api_key=os.getenv("GROQ_API_KEY")
)
```

```
# =====
# ■ STREAMING GENERATION
# =====

def generate_stream_answer(
    query: str,
    context: str
):
```

```
    prompt = f"""
You are a strict assistant.

Rules:
- Answer ONLY using the provided context
- If context is not relevant to the question,
  say: "No relevant information found"
- Do NOT guess or assume anything

Context:
{context}

Question:
{query}

Answer:
"""
```

```
stream = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ],
    stream=True
)
```

```
for chunk in stream:

    if chunk.choices[0].delta.content:

        yield chunk.choices[0].delta.content
```

```
# =====
# ■ STANDARD GENERATION
# =====

def generate_answer(
    query: str,
    context: str
):
```

```
prompt = f"""
You are a strict assistant.

Rules:
- Answer ONLY using the provided context
- If context is not relevant to the question,
  say: "No relevant information found"
- Do NOT guess or assume anything

Context:
{context}

Question:
{query}

Answer:
"""
```

```
response = client.chat.completions.create(
    model="llama-3.1-8b-instant",
    messages=[
        {
            "role": "user",
            "content": prompt
        }
    ]
)
```

```
return response.choices[0].message.content
```

7. What To Learn Next

NEXT FILE:

app.py / main.py

Reason:

Now learner understands:

- ingestion
- retrieval
- orchestration
- reranking
- LLM generation

Next:

learn how the complete RAG system is exposed as a production API using FastAPI.